

# Service Slicing e Rilevamento DoS in Reti SDN

Relazione Finale di Progetto — Architettura, Implementazione e Collaudo Sperimentale



## Autori (Gruppo di Lavoro):

- Davide Cipriano (Mat. M63001780)
- Giuseppe De Lucia (Mat. M63001783)
- Valerio Cera (Mat. M63001700)



**Esame:** Network and Cloud Infrastructures (NCI) — Università di Napoli Federico II



**Repository GitHub:**

## 1. Introduzione e Contesto Tecnologico

L'aumento continuo di servizi e delle alte prestazioni richieste di garanzie di banda e bassa latenza — come, ad esempio, nell'uso dello streaming video — rende inadeguate l'utilizzo del modello classico di rete del 'best-effort', nel quale il traffico viene trasportato in modo indifferenziato, rendendo difficile evitare la congestione causata da flussi non prioritari o da anomalie di traffico.

Il paradigma Software-Defined Networking (SDN) mira a risolvere questa criticità separando nettamente il piano di controllo (control-plane) dal piano dati (data-plane). La centralizzazione della logica di instradamento viene spostata all'interno di un controller programmabile (nel nostro caso basato sul framework Ryu e protocollo OpenFlow v1.3), questo ci permette di abilitare il meccanismo di **Network Slicing**: la suddivisione logica di un'unica infrastruttura fisica in più reti virtuali isolate, ciascuna caratterizzata da parametri di banda e priorità dedicati. Inoltre, il sistema integra un modulo di **sicurezza comportamentale per il rilevamento ed isolamento automatico di attacchi Denial of Service (DoS)**.

## 2. Obiettivi del Progetto

L'obiettivo primario del progetto è la realizzazione e sperimentazione in ambiente emulato Mininet di un'architettura SDN completa, in grado di integrare slicing reattivo e sicurezza attiva. Nello specifico:

- **Isolamento logico delle Slice:** Definizione di due flussi di rete distinte (Video e Web), ciascuna instradata su un percorso fisico separato con capacità di banda differenziata.
- **Routing Reattivo Autorizzato:** Configurazione del controller Ryu affinché instradi esclusivamente il traffico appartenente a coppie (sorgente, destinazione) esplicitamente autorizzate, scartando a priori qualsiasi altro pacchetto.
- **Classificazione dei flussi:** Riconoscimento del traffico in base a protocollo di trasporto e della porta andando ad assegnare ai traffici i relativi percorsi (UDP 9999 per il flusso Video ad alta priorità, TCP 5001 per il flusso Web).
- **Monitoraggio del Throughput in Tempo Reale:** Misurazione periodica della velocità di trasmissione dei flussi sulla slice video rilevando eventuali comportamenti anomali
- **Mitigazione Comportamentale DoS:** Identificazione delle anomalie di traffico e applicazione automatica di regole di drop temporanee (30s) nei confronti di qualsiasi host che superi la soglia prestabilita (8 Mbps).
- **Osservabilità tramite Dashboard Web:** Sviluppo di un'interfaccia di monitoraggio in tempo reale (Flask) per la visualizzazione dei grafici di traffico, dello stato degli switch e degli eventi di blocco.

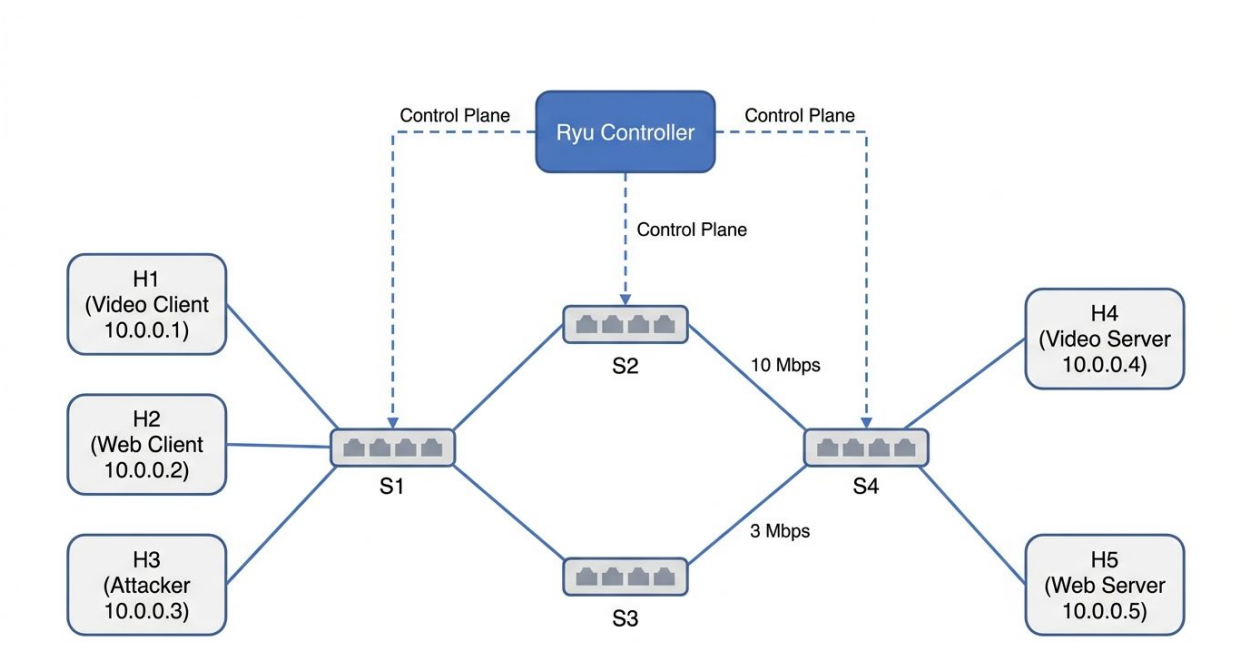
### REGOLA FONDAMENTALE DI SICUREZZA

Il rilevamento DoS è strettamente COMPORTAMENTALE e non basato sull'identità dell'host: qualunque sorgente (incluso un client legittimo come H1) che generi traffico sulla slice monitorata (video) oltre la soglia di 8 Mbps viene temporaneamente bloccata con una regola di drop ad alta priorità.

### 3. Architettura e Topologia di Rete

La topologia di rete emulata in Mininet è composta da 5 Host virtuali e 4 Open vSwitch interconnessi in modo da offrire due percorsi fisicamente distinti con capacità di banda diversa:

| Host | Ruolo / Descrizione      | Indirizzo IP | Slice & Percorso Inoltro                      |
|------|--------------------------|--------------|-----------------------------------------------|
| H1   | Client Video Legittimo   | 10.0.0.1     | Slice Video (s1 → s2 → s4, 10 Mbps, UDP 9999) |
| H2   | Client Web Legittimo     | 10.0.0.2     | Slice Web (s1 → s3 → s4, 3 Mbps, TCP 5001)    |
| H3   | Host Attacker            | 10.0.0.3     | Slice Video (s1 → s2 → s4, 10 Mbps, UDP 9999) |
| H4   | Server Video Destination | 10.0.0.4     | Destinazione Slice Video (Collegato a s4)     |
| H5   | Server Web Destination   | 10.0.0.5     | Destinazione Slice Web (Collegato a s4)       |



## 4. Implementazione del Controller Ryu

### 4.1 Service Slicing Reattivo ed Isolamento Traffico

Il controller Ryu non installa regole statiche all'avvio della rete, ma opera in modalità reattiva. Quando un primo pacchetto giunge a uno switch e non trova corrispondenze nella Flow Table (Table-Miss), viene inoltrato al controller per l'ispezione. Le slices sono definite da un dizionario con sorgente, destinazione, porta/protocollo ed un flag che stabilisce se partecipa al rilevamento DoS:

```
1  # Definiamo i flussi di rete distinte (Video e Web), ciascuno instradato su un percorso fisico distinto
2  # e definiamo i flussi che partecipano al rilevamento DoS
3
4      self.tracked_slices = {
5          "video_legit": {
6              "src": self.H1, "dst": self.H4,
7              "port_field": "udp_dst", "port": self.video_port,
8              "label": "Video (H1 -> H4)",
9              "monitor_dos": True # partecipa al rilevamento DoS
10         },
11         "attacker": {
12             "src": self.H3, "dst": self.H4,
13             "port_field": "udp_dst", "port": self.video_port,
14             "label": "Video (H3 -> H4)",
15             "monitor_dos": True # partecipa al rilevamento DoS
16         },
17         "web": {
18             "src": self.H2, "dst": self.H5,
19             "port_field": "tcp_dst", "port": self.web_port,
20             "label": "Web (H2 -> H5)",
21             "monitor_dos": False # la slice web non è soggetta a soglia DoS
22         },
23     }
```

Al primo **handshake** con lo switch installiamo la regola di default spiegata prima con priorità bassa, inoltrando il primo pacchetto al controller.

```

1  #Gestiamo l'Handshake tra switch e controller
2  @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
3  def switch_features_handler(self, ev):
4      datapath = ev.msg.datapath #switch connesso
5      parser = datapath.ofproto_parser #Il costruttore dei messaggi OpenFlow per lo switch.
6      ofproto = datapath.ofproto
7
8      self.datapaths[datapath.id] = datapath
9
10     # L'azione OFPP_CONTROLLER invia il pacchetto al controller
11     # dicendo allo switch se hai un pacchetto che corrisponde con match fai actions questo ha una priorita bassa
12     # l'azione dice di inviare il pacchetto al controller cosi lo facciamo gestire al controller (lo usiamo con i pacchetti nuovi)
13     # perche se ha registrata una regola di instradamento a priorita piu alta la segue
14
15     match = parser.OFPMatch() #match vuoto (cattura qualsiasi pacchetto senza regola specifica)
16
17     actions = [
18         parser.OFPActionOutput(ofproto.OFPP_CONTROLLER, ofproto.OFPCML_NO_BUFFER)
19     ]
20
21     # installa una la regola con priority=0
22     self.add_flow(datapath, priority=0, match=match, actions=actions)
23     self.logger.info("Switch connesso: %s", datapath.id)
24

```

Quando il pacchetto sconosciuto viene inviato al controller, viene invocata la funzione **packet\_in\_handler**, che a sua volta invoca funzioni diverse a seconda del protocollo del pacchetto (ARP, IP).

```

1  # Viene eseguita ogni volta che uno switch riceve un pacchetto "nuovo" e lo invia al controller per chiedere istruzioni
2  @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
3  def packet_in_handler(self, ev):
4
5      msg = ev.msg # Estrae l'oggetto messaggio OpenFlow contenuto nell'evento ev
6      datapath = msg.datapath
7      in_port = msg.match["in_port"] #Legge la porta di ingresso dello switch attraverso la quale il pacchetto è entrato fisicamente
8
9      pkt = packet.Packet(msg.data)
10     eth = pkt.get_protocol(ethernet.ethernet)
11
12     # se il pacchetto viene usato per scoperta topologica (LLDP) lo ignoriamo facendo return
13     if eth.ethertype == ether_types.ETH_TYPE_LLDP:
14         return
15
16     # caso in cui il protocollo è ARP e inoltra al metodo per la risoluzione indirizzi (handle_arp)
17     arp_pkt = pkt.get_protocol(arp.arp)
18     if arp_pkt:
19         self.handle_arp(datapath, pkt, eth, arp_pkt, in_port)
20         return
21
22     # caso in cui il protocollo è IP gestisce il pacchetto con handle_ipv4
23     ip_pkt = pkt.get_protocol(ipv4.ipv4)
24     if ip_pkt:
25         self.handle_ipv4(datapath, pkt, eth, ip_pkt, in_port)
26         return
27

```

## 4.2 Rilevamento Comportamentale e Mitigazione DoS

Il thread di monitoraggio del controller interroga periodicamente gli switch (ogni 5s) andando a calcolare il throughput dei flussi sulla slice video. Per ottimizzare il conteggio ed evitare misurazioni duplicate lungo il cammino, la misurazione del throughput viene eseguita esclusivamente sullo switch di uscita **s4 (MEASUREMENT\_DPID = 4)**.

Il rilevamento DoS è comportamentale e non basato sull'identità dunque qualsiasi host che abbia `monitor_dos=true` e superi la soglia di 8Mbps viene bloccato

```
1  # mitigazione e blocco dei pacchetti
2  def block_host(self, info, current_mbps):
3      host_ip = info["src"]
4      now = time.time()
5
6      if host_ip in self.blocked_hosts:
7          if now < self.blocked_hosts[host_ip]:
8              return
9
10     self.blocked_hosts[host_ip] = now + self.block_timeout
11
12     #associare il corretto numero di protocollo IP, UDP
13     ip_proto = 17 if info["port_field"] == "udp_dst" else 6
14
15     # installa le regole di drop per tutti gli switch
16     for datapath in self.datapaths.values():
17         parser = datapath.ofproto_parser # costruttore degli oggetti e dei messaggi OpenFlow
18
19         match = parser.OFPMatch(
20             eth_type=ether_types.ETH_TYPE_IP,
21             ipv4_src=host_ip,
22             ipv4_dst=info["dst"],
23             ip_proto=ip_proto,
24             **{info["port_field"]: info["port"]}
25         )
26
27         self.add_drop_flow(
28             datapath=datapath,
29             match=match,
30             hard_timeout=self.block_timeout
31         )
```

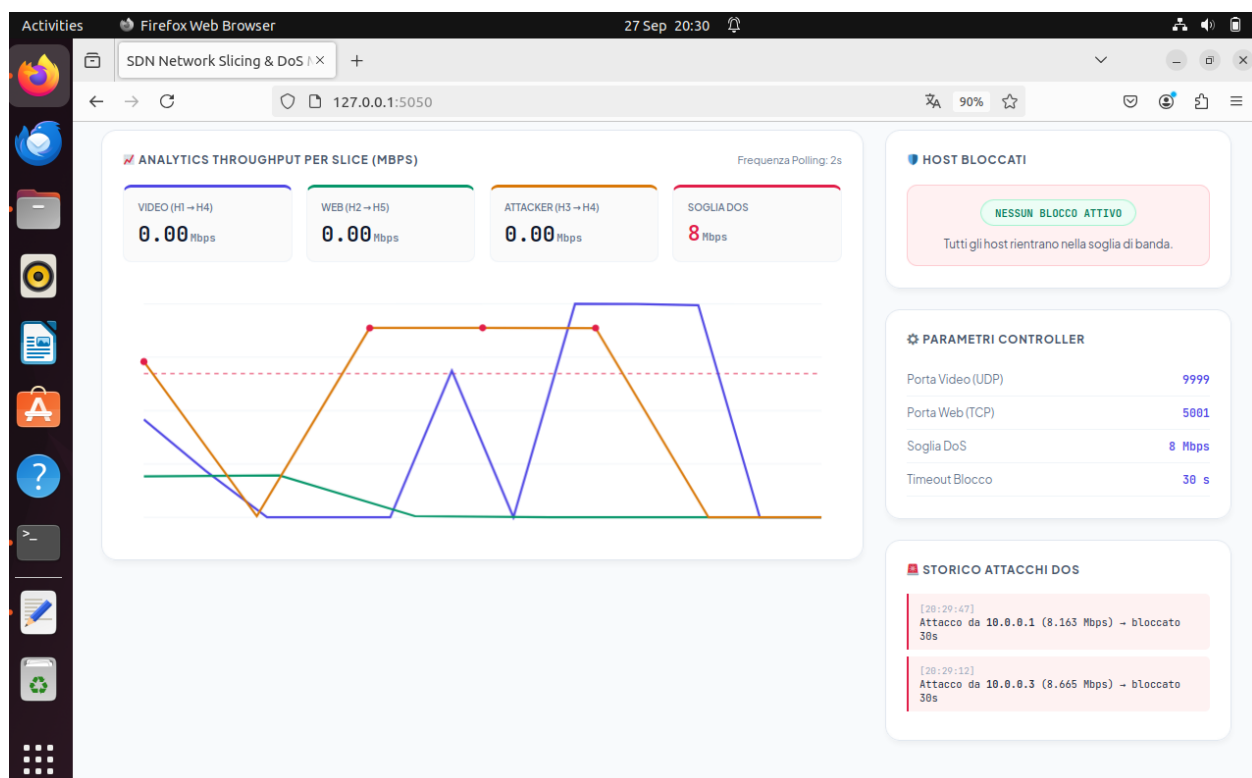
I parametri operativi sono definiti all'interno del file di configurazione **config.json**:

| Parametro JSON            | Valore Default | Descrizione e Ruolo Operativo                                  |
|---------------------------|----------------|----------------------------------------------------------------|
| <b>video_port</b>         | 9999           | Porta UDP utilizzata per la classificazione della Slice Video. |
| <b>web_port</b>           | 5001           | Porta TCP utilizzata per la classificazione della Slice Web.   |
| <b>dos_threshold_mbps</b> | 8 Mbps         | Soglia di bit-rate oltre la quale scatta il blocco automatico. |
| <b>block_timeout</b>      | 30 secondi     | Durata del blocco applicato alle regole di drop.               |

### 4.3 Dashboard di Monitoraggio Web in Tempo Reale

Per analizzare l'andamento del traffico nella rete abbiamo sviluppato una dashboard web. A livello architetturale, il controller scrive periodicamente lo stato della rete su un file *stats.json* mediante una tecnica di scrittura atomica (evitando così il rischio di letture parziali). Un'applicazione Flask indipendente (*dashboard.py*) legge questi dati e li espone dinamicamente su una pagina web (<http://127.0.0.1:5050>) con un refresh ogni 2 secondi, all'interno della quale mostriamo:

- **Grafico Live:** mostriamo tre flussi di traffico (Video , Web , Video hacker) e la soglia DoS.
- **Topologia:** lo stato e il numero dei controller e switch connessi
- **Lista DoS:** Una lista degli Host bloccati in tempo reale e il tempo rimanente per sbloccare l'host.
- **Storico DoS:** Una lista contenente tutti gli eventi DoS rilevati



## 5. Test e Risultati

La validazione dell'architettura è stata condotta eseguendo una suite composta da 8 test diretti a verificare il corretto instradamento su ciascuna slice, l'isolamento tra le slice, rilevamento e risposta al traffico anomalo:

| # | Obiettivo Test           | Comando di Esecuzione                                                   | Esito Osservato                                  | Stato |
|---|--------------------------|-------------------------------------------------------------------------|--------------------------------------------------|-------|
| 1 | Slice Video Legittima    | h1 ping -c 3 h4                                                         | 3/3 ricevuti, 0% packet loss                     | PASS  |
| 2 | Isolamento Web → Video   | h2 ping -c 3 h4                                                         | 0/3 ricevuti, 100% packet loss                   | PASS  |
| 3 | Slice Web Legittima      | h5 iperf -s -p 5001 &<br>h2 iperf -c 10.0.0.5 -p 5001 -t 10             | Trasferimento riuscito (~2.5 Mbps su link da 3M) | PASS  |
| 4 | Isolamento Video → Web   | h1 ping -c 3 h5                                                         | 0/3 ricevuti, 100% packet loss                   | PASS  |
| 5 | Traffico Video Legittimo | h4 iperf -s -u -p 9999 &<br>h1 iperf -c 10.0.0.4 -u -p 9999 -b 5M -t 10 | Instradato correttamente, sotto soglia DoS       | PASS  |
| 6 | Rilevamento DoS (h3)     | h3 iperf -c 10.0.0.4 -u -p 9999 -b 20M -t 20                            | Throughput > 8 Mbps: h3 bloccato per 30s         | PASS  |
| 7 | Rilevamento DoS (h1)     | h1 iperf -c 10.0.0.4 -u -p 9999 -b 20M -t 20                            | Throughput > 8 Mbps: h1 bloccato per 30s         | PASS  |
| 8 | Dashboard Web            | python3 dashboard.py<br>(http://127.0.0.1:5050)                         | Visualizzazione live di flussi, soglia e blocchi | PASS  |

## 6. Conclusioni

Il lavoro svolto dimostra con successo l'efficacia dell'integrazione tra **Service Slicing reattivo** e **mitigazione comportamentale del traffico anomalo DoS** all'interno di un'architettura SDN centralizzata. La separazione concettuale tra piano di controllo (Ryu), simulazione di rete (Mininet) ed osservabilità (Dashboard Flask) assicura robustezza e modularità.

L'utilizzo di un approccio comportamentale rende il sistema di rilevamento DoS efficace anche in contesti operativi realistici, senza richiedere la conoscenza preventiva delle sorgenti malevole.